HW3 - STAT 701

Wells Guan

Date: October 28, 2026

Preface: for all multitaper/DPSS related questions, you are welcome to use the R package `multitaper` (or python/Julia equivalents). You are also welcome to use chatbots or other external tools for hard Fourier integrals. It is not my intention for you to spend hours grinding out $\int_{-1/2}^{1/2} e^{i\omega h} e^{-\theta|\omega|}\, \mathrm{d}\omega$.

Problem 1

Consider a sample path of length n from a zero-mean, stationary time series $\{X_t\}_{t=1}^n$. Define its weighted Discrete Fourier Transform (DFT) at the Fourier frequencies $f_k = k/n$ for $k \in \{-[(n-1)/2], \ldots, [n/2]\}$, where $[\cdot]$ denotes the integer part of a number, by

$$\widetilde{X}(f_k) = \sum_{t=1}^{n} w_t X_t e^{-i2\pi f_k t}$$

for a sequence of real-valued data taper weights $\{w_t\}_{t=1}^n$.

1. Let $w_t = n^{-1/2}$ for all t, so that $|\widetilde{X}(f_k)|^2 = I(f_k)$ is the standard un-tapered periodogram estimator. Pick $n = 512$ and a known autocovariance function $\gamma(h)$ for your process (e.g., an ARMA(p,q)). Using simulation:
   (a) Draw $m \gg n$ independent sample paths (e.g., $m \approx 10n$).
   (b) Compute the vector of DFT coefficients $\widetilde{\mathbf{X}}^{(i)} = [\widetilde{X}^{(i)}(f_k)]$ for Fourier frequency f_k defined above for each realization $i = 1, \ldots, m$.
   (c) Construct the empirical covariance matrix $\widehat{\Sigma} \in \mathbb{C}^{n\times n} = \sum_{i=1}^m \widetilde{\mathbf{X}}^{(i)} \widetilde{\mathbf{X}}^{(i)H}$ of these DFT coefficients.

   Inspect the matrix $\widehat{\Sigma}$ and comment on the resulting structure: how correlated are the estimates across different Fourier frequencies f_k and $f_{k'}$? If you were interested to make a diagonal approximation of $\widehat{\Sigma}$, what information would you be throwing away?

   Sol.

2. Repeat this exercise using a normalized Slepian/discrete prolate spheroidal sequence (DPSS) taper sequence $w_t = h_t^{(0)}$ with time-bandwidth parameter $NW = 10$, which you can obtain with the function `dpss(n, 1, 10)$v` from the R multitaper package (there are also a few Julia packages that offer this, shoot me a note if the first one you try doesn't instantly work). What structural differences do you observe in the empirical covariance matrix compared to the un-tapered case?

   Sol.

3*. Analytically derive the cross-covariance $\mathrm{Cov}(\widetilde{X}(f_k), \widetilde{X}(f_{k'}))$ for the flat weight sequence $w_t = n^{-1/2}$ in terms of the process spectral density $S(f)$ or autocovariance $\gamma(h)$. Simplify as much as possible using Dirichlet kernel properties. Based on your result, at what rate do you think these covariances go to zero?

Sol.

Problem 2

Let's explore the local versus broadband bias tradeoffs in spectral density estimation.

1. Write code that uses circulant embedding to *exactly* simulate a length-n sample path from a stationary time series with arbitrary covariance function $\gamma(\cdot)$. Please design its interface to accept an ACF sequence $\{\gamma(0), \ldots, \gamma(n-1)\}$ and return the length-n sequence. You may use a built-in library for the FFT, but please do the other book-keeping yourself.

   Sol.

2. Recall the periodogram and the multitaper estimator described in class. Please design spectral densities/covariance functions to achieve the following failure modes, and demonstrate them by simulating data and computing the estimators:
   (a) The multitaper estimator misses two distinct peaks and only reports one, but the periodogram gets them both.

   Sol.

   (b) The periodogram loses a faint spectral peak, but the multitaper catches it.

   Sol.

   R functions like `spec.pgram` and `spec.mtm`, the latter being from the `multitaper` package, may be helpful. In both cases, you have multiple options for achieving this. You could design an ARMA process, but you may find it easier to design the *spectrum* that makes the pathology obvious and then back out the ACF from it. *You are welcome to use chatbots to solve hard integrals. But please be disciplined about restricting your use of them to just the hard integrals!*

3. A *harmonic* of a fundamental frequency f_0 is an integer multiple $k f_0$ for $k \in \{2, 3, 4, \ldots\}$. Sometimes simple nonlinear transformations or other irregularities can cause your SDF estimator to also have peaks at these harmonics of "true" peaks. Let $n = 2048$ or something like that, and consider the time series

$$X_t = \tanh\big(\alpha \cos(2\pi f_0 t + \phi)\big) + Z_t,$$

   where Z_t is a white noise with variance σ^2, $f_0 = 0.05$, and $\phi \sim \mathrm{Unif}(0, 2\pi)$. Simulate such an X_t.

Does it look like it could be a stationary process? Look at a spectral density estimator for it as if it were, and play with values of α to make the harmonics more or less prominent. Then find another nonlinear function besides tanh to apply to your periodic function that also induces spurious harmonics in your estimator. Try to pick something interesting that gives you stationary-looking data!

Sol.

4. Pull the tide gauge data from Canvas. Pick a segment of moderate length (at least a few thousand points) with no gaps, and compute several spectral density estimators: a periodogram, a sine multitaper, a DPSS multitaper, and whatever else your heart desires. Inspect each of them and try to answer: how many of these peaks are real, and how many are harmonics? How would you model this spectral density? You might consider taking a look at a list of lunar frequencies, which you can find on, e.g., Wikipedia. You don't have to include an answer to this part of the question, but try to ponder: how might you distinguish that non-linear mean from structure in a stationary covariance function?

   Sol.

Problem 3

Write code to implement the *Whittle likelihood* described in class. In particular, please write a function that takes two arguments: the (potentially weighted) DFT of your data, and SDF parameters.

1. Pull up the NAO data and take the first 1024 measurements. Look at an exploratory estimator for the spectrum (perhaps a multitaper…) and find a functional form you like for the spectrum. This should be something involving parameters, like $S_\theta(\omega) = \theta_0 e^{-\theta_1|\omega|}$ or something. Discuss your choices.

   Sol.

2. By specifying your Whittle likelihood and handing that function to an optimizer (be careful about bound constraints for valid parameters!), fit that functional form to the data.

   Sol.

3*. Using a numerical integrator or a chatbot or however you please, get from the spectral density to obtaining the corresponding ACF values $\{\gamma(0), \ldots, \gamma(n-1)\}$. Now take your fitted model SARIMA model from last week and evaluate the true likelihood, involving the covariance matrix and stuff, for each fitted model. Which of the two models has a better likelihood? Try and investigate what you think is going on and why one or the other was better.

   Sol.

A Python code

The following files are templates to be completed. Data are stored in `data/`, and figures should be saved in `png/`.

A.1 Problem 1

```python
from pathlib import Path
from statsmodels.tsa.arima_process import arma_generate_sample

import numpy as np
import matplotlib.pyplot as plt

ROOT = Path(__file__).resolve().parents[1]
DATA_DIR = ROOT / "data"
FIGURE_DIR = ROOT / "png"
SEED = 70103

N = 512
M = 10 * N
NW = 10


def simulated_paths(times_series, alpha = 10, rng=None):
    n = times_series.shape[0]
    weights = np.ones(n)/(n ** 0.5)
    frequencies = np.arange(-np.floor((n-1)/2),np.floor(n/2)+1)/n
    dft = np.stack([weighted_dft(times_series, weights, frequency) for frequency in frequencies])
    return dft

def draw_paths(paths, x=None, component="real", ax=None):
    paths = np.asarray(paths)
    if paths.ndim != 2 or 0 in paths.shape:
        raise ValueError("paths must be a nonempty (m, n) array")
    if component == "real":
        values = paths.real
    elif component == "imag":
        values = paths.imag
    elif component == "abs":
        values = np.abs(paths)
    else:
        raise ValueError("component must be 'real', 'imag', or 'abs'")

    m, n = paths.shape
    x = np.arange(1, n + 1) if x is None else np.asarray(x)
    if x.ndim != 1 or x.size != n:
        raise ValueError("x must be a one-dimensional array of length n")
    if ax is None:
        _, ax = plt.subplots(figsize=(10, 5))

    colors = plt.get_cmap("viridis")(np.linspace(0.05, 0.95, m))
    lines = ax.plot(x, values.T, linewidth=0.9, alpha=0.8 if m <= 20 else 0.25)
    for index, (line, color) in enumerate(zip(lines, colors), start=1):
```

```
        line.set_color(color)
        line.set_label(f"Series {index}")
    ax.set_xlabel("Index")
    ax.set_ylabel({"real": "Real part", "imag": "Imaginary part", "abs": "Magnitude"}[component])
    ax.grid(alpha=0.2)
    if m <= 10:
        ax.legend()
    return ax



def draw_matrix(matrix, component="abs", ax=None, cmap=None, colorbar=True, gamma=0.4):
    from matplotlib.colors import PowerNorm

    if not np.isfinite(gamma) or gamma <= 0:
        raise ValueError("gamma must be a positive finite number")
    matrix = np.asarray(matrix)
    if matrix.ndim != 2 or 0 in matrix.shape:
        raise ValueError("matrix must be a nonempty two-dimensional array")
    if component == "abs":
        values = np.abs(matrix)
    elif component == "real":
        values = matrix.real
    elif component == "imag":
        values = matrix.imag
    else:
        raise ValueError("component must be 'abs', 'real', or 'imag'")
    if not np.isfinite(values).all():
        raise ValueError("matrix values must be finite")
    if ax is None:
        _, ax = plt.subplots(figsize=(8, 6))

    options = {}
    if component == "abs":
        cmap = "viridis" if cmap is None else cmap
        limit = float(np.max(values)) or 1.0
        options["norm"] = PowerNorm(gamma=gamma, vmin=0, vmax=limit)
    else:
        cmap = "RdBu_r" if cmap is None else cmap
        limit = float(np.max(np.abs(values))) or 1.0
        options.update(vmin=-limit, vmax=limit)
    image = ax.imshow(values, origin="upper", interpolation="nearest",
                      aspect="auto", cmap=cmap, **options)
    ax.set_xlabel("Column index")
    ax.set_ylabel("Row index")
    if colorbar:
        bar = ax.figure.colorbar(image, ax=ax)
        bar.set_label({"abs": "Magnitude", "real": "Real part", "imag": "Imaginary part"}[component])
    return ax


def weighted_dft(paths, weights, frequency):
    time = np.arange(1,paths.shape[0]+1)
    exp_term = np.exp( - 1j * 2 * np.pi * frequency * time)
```

```
    return np.sum(weights * paths * exp_term)


def empirical_dft_covariance(coefficients):
    dft_covaiance = np.outer(coefficients[0], coefficients[0].conj())
    for coefficient in coefficients[1:]:
        dft_covaiance += np.outer(coefficient, coefficient.conj())
    return dft_covaiance


def main():
    FIGURE_DIR.mkdir(parents=True, exist_ok=True)
    rng = np.random.default_rng(SEED)

    samples = np.stack([arma_generate_sample(
    ar=[1, -0.6],
    ma=[1, 0.3],
    nsample=N,
    scale=1,
    burnin=500,
    distrvs=rng.standard_normal,
    ) for i in range(M)])

    #fig, ax = plt.subplots()

    #ax = draw_paths(sample[:10], ax = ax)

    #ax.set_title("ARMA sample paths")
    #ax.set_xlabel("Time")
    #ax.set_ylabel("Value")
    #ax.figure.savefig(FIGURE_DIR / "sample_paths.png", bbox_inches="tight")

    #plt.show()

    dft = np.stack([simulated_paths(sample) for sample in samples])

    empirical_covariance = empirical_dft_covariance(dft)

    fig, ax = plt.subplots()

    ax = draw_matrix(empirical_covariance, ax = ax)

    ax.set_title("ARMA DFT Empirical Covaiance Matrix")
    ax.figure.savefig(FIGURE_DIR / "empirical_covariance_matrix.png", bbox_inches="tight")

    plt.show()

if __name__ == "__main__":
    main()
```

A.2 Problem 2

```python
from pathlib import Path

import numpy as np

ROOT = Path(__file__).resolve().parents[1]
DATA_DIR = ROOT / "data"
FIGURE_DIR = ROOT / "png"
SEED = 70103

TIDE_DATA_PATH = DATA_DIR / "tide.csv"


def circulant_embedding(acf, rng=None):
    raise NotImplementedError


def periodogram(series):
    raise NotImplementedError


def multitaper_spectrum(series, tapers):
    raise NotImplementedError


def compare_peak_resolution(rng):
    raise NotImplementedError


def compare_faint_peak_detection(rng):
    raise NotImplementedError


def explore_harmonics(rng):
    raise NotImplementedError


def analyze_tide_data(path=TIDE_DATA_PATH):
    raise NotImplementedError


def main():
    FIGURE_DIR.mkdir(parents=True, exist_ok=True)
    rng = np.random.default_rng(SEED)
    raise NotImplementedError


if __name__ == "__main__":
    main()
```

A.3 Problem 3

```python
from pathlib import Path

import numpy as np

ROOT = Path(__file__).resolve().parents[1]
DATA_DIR = ROOT / "data"
FIGURE_DIR = ROOT / "png"
SEED = 70103

NAO_DATA_PATH = DATA_DIR / "nao.csv"
N = 1024


def load_nao(path=NAO_DATA_PATH, n=N):
    raise NotImplementedError


def spectral_density(frequencies, parameters):
    raise NotImplementedError


def whittle_likelihood(dft, parameters):
    raise NotImplementedError


def fit_spectrum(dft):
    raise NotImplementedError


def spectrum_to_acf(parameters, n=N):
    raise NotImplementedError


def gaussian_log_likelihood(series, acf):
    raise NotImplementedError


def main():
    FIGURE_DIR.mkdir(parents=True, exist_ok=True)
    raise NotImplementedError


if __name__ == "__main__":
    main()
```